

Enunciado

Se pide generar un conjunto de $N = 20$ partículas en 1D con densidad promedio $\bar{\rho}$ y añadir una perturbación sinusoidal pequeña $\epsilon \sin(2\pi x_0)$. Se pide la evolución temporal y mostrar gráficamente cómo se acumulan partículas en algunas zonas. Ayuda: se puede dividir el segmento en 10 compartimentos y ver cómo se van acumulando en algunos y otros se vacían.

Estamos simulando un fenómeno físico y geométrico. Este es el modelo base que usan los astrofísicos para explicar cómo se formaron las galaxias en el universo (se conoce como la Aproximación de Zel'dovich en 1D).

Podremos prever como será la acumulación de partículas si estudiamos el seno dentro de la componente velocidad.

- Mitad izquierda (de $x = 0$ a $x = 0.5$): El seno es positivo. Esto significa que las partículas que nacieron en esta mitad reciben un empujón hacia la derecha ($-->$).
- Mitad derecha (de $x = 0.5$ a $x = 1$): El seno es negativo. Las partículas que nacieron aquí reciben un empujón hacia la izquierda ($<--$).
- Puntos críticos ($x = 0.25$ y $x = 0.75$): Es donde el empujón es más fuerte (velocidad máxima).
- Puntos de anclaje ($x = 0$ y $x = 0.5$): El seno vale cero. Las partículas que nacieron exactamente ahí no se mueven.

El sistema podría ser visto como un anillo 1 D en el que tenemos puntos confinados dentro de este (estos están equiespaciados inicialmente). A estos puntos se les da una velocidad inicial en cierto sentido dependiendo de su posición (en función del seno) y a continuación estudiaremos su evolución dadas estas condiciones.

```
In [10]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
```

```
In [ ]: #Parametros
N = 20 # Numero de puntos
epsilon = 0.05
pi = np.pi
xi_val = []
# Generar los datos
x0 = np.linspace(0, 1, N, endpoint=False)

for t in range(0,100):

    #Condiciones de frontera
    xi = (x0 + epsilon * np.sin(2*pi*x0) * (t**(2/3))) % 1.0 # el módulo con número

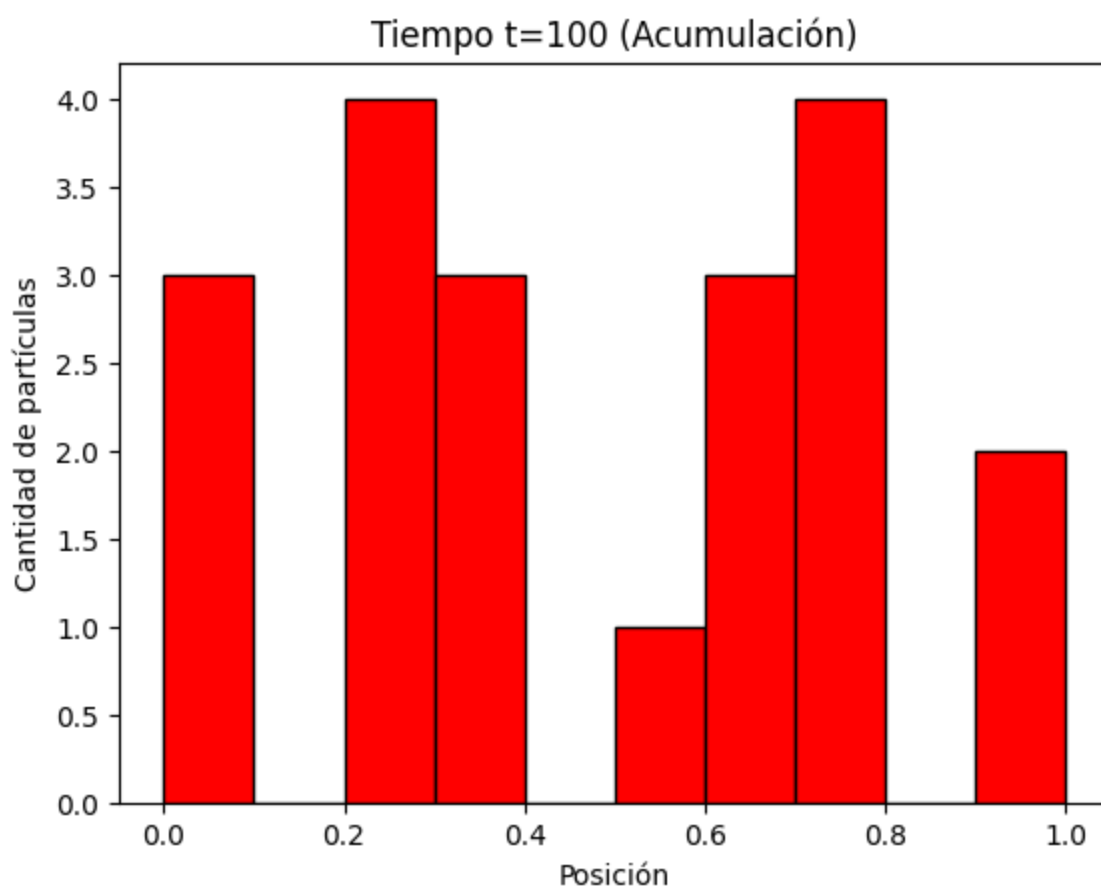
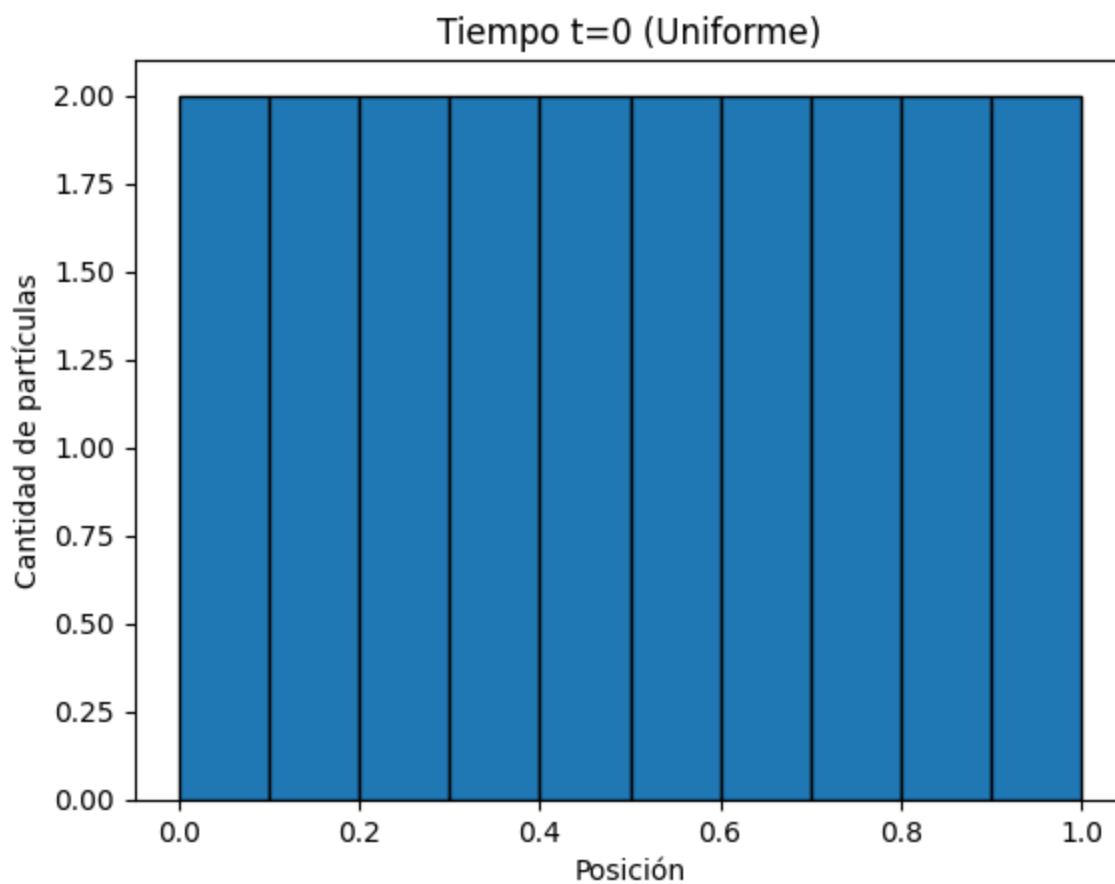
    # Epsilon * sin(2pi*x0) es una velocidad inicial que se le añade al sistema par

    xi_val.append(xi)

# Gráfico para el Tiempo t=0 (Estado inicial)
plt.figure(1)
plt.title("Tiempo t=0 (Uniforme, visión EULERIANA)")
plt.xlabel("Posición")
plt.ylabel("Cantidad de partículas")
plt.hist(xi_val[0], bins=10, range=(0, 1), edgecolor='black')

# Gráfico para el Tiempo t=9 (Estado final)
plt.figure(2)
plt.title("Tiempo t=100 (Acumulación, visión EULERIANA)")
plt.xlabel("Posición")
plt.ylabel("Cantidad de partículas")
plt.hist(xi_val[-1], bins=10, range=(0, 1), edgecolor='black', color='red')

plt.show()
```



Interpretación resultados

En las dos gráficas que hemos impreso vemos la evolución temporal de las partículas en nuestro círculo 1D en distintos instantes del tiempo. En el tiempo 0, en el histograma azul vemos como cada partícula esta distribuida equiespacialmente a lo largo del anillo teniendo una densidad homogénea. En el tiempo 30, vemos como las partículas han acabado colapsando hacia el centro del anillo (0.5) dejando una densidad heterogénea con menos partículas en los bordes y más en el medio.

Gráfico adicional

En el siguiente gráfico enseñaremos la distribución de cada partícula en cada instante en un espacio 2D

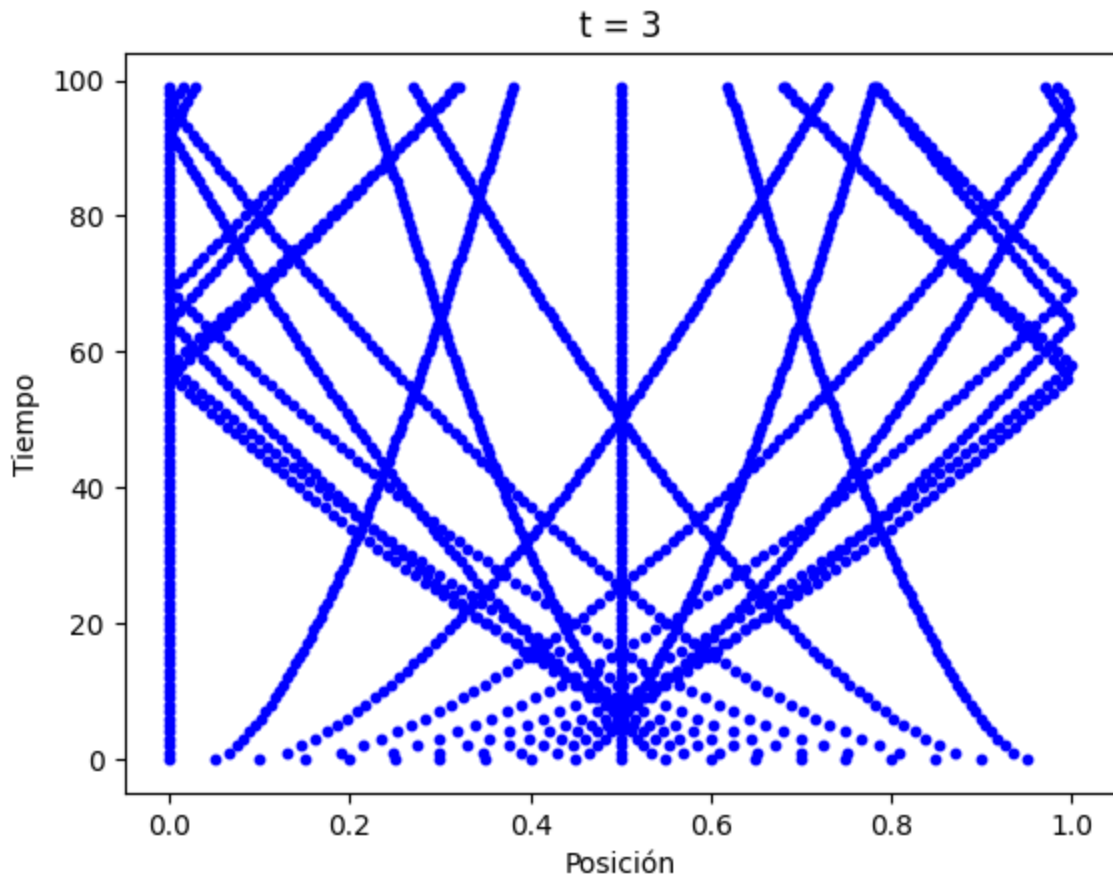
```
In [ ]: #Parametros
N = 20 # Numero de puntos
epsilon = 0.05
pi = np.pi
xi_val = []
# Generar Los datos
x0 = np.linspace(0, 1, N, endpoint=False)

for t in range(0,100):

    #Condiciones de frontera
    xi = (x0 + epsilon * np.sin(2*pi*x0) * (t**(2/3))) % 1.0 # el módulo con número

    # Epsilon * sin(2pi*x0) es una velocidad inicial que se le añade al sistema par

    plt.scatter(xi, [t]*N, color='blue', s=10)
    plt.title("t = 100 (Visión euleriana)")
    plt.xlabel("Posición")
    plt.ylabel("Tiempo")
plt.show()
```



Interpretación

El sistema experimenta un movimiento inercial puro (velocidades constantes dictadas por la perturbación inicial senoidal). Debido a esto, las partículas cruzan la zona central ($x = 0.5$) formando una caústica y continúan su trayectoria. Sin embargo, debido a la topología compacta del espacio, las partículas que se 'escapan' darán la vuelta al dominio y volverán a colisionar en $x = 0$ y $x = 1$ en tiempos posteriores, creando un ciclo repetitivo de formación de caústicas y vacíos puramente cinemático, sin necesidad de fuerzas de restauración.

```
In [11]: # 1. PARÁMETROS INICIALES
N = 20 # Número de partículas
epsilon = 0.05
frames = 150 # Cantidad de "fotos" para la animación
t_max = 40 # Tiempo máximo (más largo para ver el efecto de  $t^{(2/3)}$ )
t_vals = np.linspace(0, t_max, frames) # Creamos un tiempo continuo

# 2. CALCULAR TODA LA FÍSICA (Aproximación de Zel'dovich Cosmológica)
x0 = np.linspace(0, 1, N, endpoint=False)
xi_val = []

for t in t_vals:
    # Usamos  $t^{(2/3)}$  por la ecuación diferencial del universo dominado por materia
    xi = (x0 + epsilon * np.sin(2 * np.pi * x0) * (t**(2/3))) % 1.0
    xi_val.append(xi)
```

```

# 3. PREPARAR EL LIENZO (Dos gráficos juntos)
# gridspec_kw hace que el gráfico de arriba sea más alto que el de abajo
fig, (ax_hist, ax_line) = plt.subplots(2, 1, figsize=(9, 6), gridspec_kw={'height_r
fig.subplots_adjust(hspace=0.4)

# --- Configurar Histograma (Euleriano) ---
ax_hist.set_xlim(0, 1)
ax_hist.set_ylim(0, 12) # Altura máxima de las barras
ax_hist.set_ylabel("Densidad (Número de Partículas)")
ax_hist.set_title("Visión Euleriana (Estructura a Gran Escala)", fontsize=11)
counts, edges = np.histogram(xi_val[0], bins=10, range=(0,1))
bars = ax_hist.bar(edges[:-1], counts, width=0.1, align='edge', edgecolor='black',

# --- Configurar Línea 1D (Lagrangiano) ---
ax_line.set_xlim(0, 1)
ax_line.set_ylim(-1, 1)
ax_line.get_yaxis().set_visible(False) # Ocultamos el eje Y porque solo se mueven e
ax_line.set_xlabel("Espacio Comóvil (x)")
ax_line.set_title("Visión Lagrangiana (Partículas Individuales)", fontsize=11)
ax_line.axhline(0, color='black', linewidth=1) # Dibujar la "carretera"
scatter = ax_line.scatter(xi_val[0], np.zeros(N), color='red', s=70, edgecolor='dar

# 4. LA FUNCIÓN MÁGICA (Actualiza los datos en cada frame)
def update(frame):
    # Actualizar la altura de las barras del histograma
    counts, _ = np.histogram(xi_val[frame], bins=10, range=(0,1))
    for bar, count in zip(bars, counts):
        bar.set_height(count)

    # Actualizar la posición de las partículas rojas en la línea
    nuevas_posiciones = np.c_[xi_val[frame], np.zeros(N)]
    scatter.set_offsets(nuevas_posiciones)

    # Actualizar el título principal con el reloj cósmico
    fig.suptitle(f"Evolución Cosmológica - Tiempo: {t_vals[frame]:.1f}", fontsize=1
    return bars.patches + [scatter]

# 5. CREAR LA ANIMACIÓN
# interval=50 son los milisegundos que dura cada frame
ani = FuncAnimation(fig, update, frames=frames, interval=50, blit=False)

# Cerramos el "plot" estático para que no se dibuje doble en Jupyter
plt.close()

# 6. MOSTRAR COMO VIDEO INTERACTIVO EN JUPYTER
HTML(ani.to_jshtml())

```

Out[11]:

